India Tourism Water Footprint Pipeline

Step-by-Step Methodology with Formulas

A complete technical walkthrough of all 9 pipeline modules

What this pipeline does

Given: India's Supply-Use Tables (2015-16, 2019-20, 2021-22) + Tourism Satellite Account + EXIOBASE water data
Answer: How many cubic metres of freshwater does India's tourism sector consume — directly and through its entire supply chain?

The answer requires 9 sequential calculation steps, each building on the outputs of the previous.
This document explains every step, every formula, and every number it produces.

The 9-Step Pipeline at a Glance

Step	Module	What It Does	Key Formula / Output
1	`build_io_tables.py`	Convert raw Supply-Use Tables into IO tables	$L = (I - A)^{-1}$ where $A = Z/x$
2	`build_water_coefficients.py`	Extract water use per ₹ crore from EXIOBASE	$W_i = F_{water_i} / x_i$ (m^3/₹ cr)
3	`build_tourism_demand.py`	Build tourism demand vectors for each year	$Y_{nominal} = TSA_{base} \times NAS_{growth} \times CPI$
4	`calculate_indirect_twf.py`	Core: compute indirect water footprint	$TWF = W \cdot L \cdot Y$ (bn m^3)
5	`calculate_direct_twf.py`	Activity-based operational water (hotels, transport)	Direct = rooms × L/room/night + meals × L/meal
6	`outbound_twf.py`	Outbound TWF + net India water balance	Outbound = N × days × (WF/365) × 1.5
7	`calculate_sda_mc.py`	SDA decomposition + Monte Carlo uncertainty	$\Delta TWF = \Delta W \cdot L \cdot Y + W \cdot \Delta L \cdot Y + W \cdot L \cdot \Delta Y$
8	`visualise_results.py`	Generate all charts (waterfall, Sankey, etc.)	Outputs: Figures 1–8
9	`compare_years.py`	Cross-year report + fill report_template.md	Final Markdown/HTML report

STEP 1 Build Input–Output Tables [build_io_tables.py]

The pipeline starts with India's **Supply-Use Tables (SUTs)** — official MoSPI data showing, for 140 product types, how much each industry buys from every other (intermediate flows), and how much goes to final consumers. We transform these into a compact **symmetric product×product Input-Output table**, which is the mathematical foundation every subsequent step needs.

What are Supply-Use Tables?

A Supply-Use Table has two halves:

- The Supply Table (V): what each industry produces. Rows = industries, columns = products.
- The Use Table (U): what each industry buys. Rows = products, columns = industries.

Together they describe the entire flow of goods and services in the Indian economy for a fiscal year.

The Product Technology Assumption (PTA)

To go from SUTs to a symmetric IO table, we apply the PTA — the standard method used by OECD and Eurostat. The assumption is: each product is always produced with the same technology, regardless of which industry produces it. This gives us four matrices:

```
Step 1.1:  D = V / q          (Market-share matrix, products × industries)
           D[i,j] = share of product i output made by industry j

Step 1.2:  Z = U @ D.T        (Symmetric intermediate flow matrix, products
× products)
           Z[i,j] = ₹ crore of product i used to produce all output of
product j

Step 1.3:  A = Z / x          (Technical coefficient matrix, products ×
products)
           A[i,j] = ₹ worth of product i needed per ₹ of product j output

Step 1.4:  L = (I − A)⁻¹     (Leontief inverse — the KEY matrix)
           L[i,j] = total direct + indirect requirement of product i
                    per unit of final demand for product j
```

Why the Leontief Inverse matters

L is not just a mathematical convenience. Each element L[i,j] tells you:
'To satisfy one rupee of final demand for product j, how many rupees of product i does the ENTIRE economy need to produce (counting all supply chain layers)?'

Example: L[paddy rice, hotels] = 1.42 means: for every ₹1 tourists spend at hotels, ₹1.42 worth of paddy rice production is ultimately required across the supply chain — even though tourists never directly buy rice.

This is why tourism has a water footprint far larger than the hotels can see.

Quality Checks Built In

Check	Formula / Condition	Why It Matters
Hawkins-Simon condition	Spectral radius $\rho(A) < 1$	Guarantees $L = (I-A)^{-1}$ is non-negative. If any sector needed more inputs than it produces, the inverse would blow up.
IO balance error	|total output − total input| / output < 1%	Verifies SUT data integrity. The 2021-22 preliminary data has minor MoSPI discrepancies.
Negative output repair	$x < 0 \rightarrow \|x\|$; A column sums ≥ 1 → rescale to 0.95	2022 crude petroleum data error: intermediate inputs exceed total output by 55%. Without this fix, the A matrix is invalid.
Real output deflation	x_real = x_nominal / CPI	Isolates genuine volume change from inflation. Needed for inter-year intensity comparisons.

Outputs from Step 1

- io_Z_{year}.csv — 140×140 intermediate flow matrix Z (₹ crore)
- io_A_{year}.csv — 140×140 technical coefficient matrix A (dimensionless)
- io_L_{year}.csv — 140×140 Leontief inverse L (dimensionless)
- io_output_{year}.csv — total output x, final demand, real output

STEP 2 Build Water Coefficients [build_water_coefficients.py]

We need to know: **how much water does each sector use per ₹ crore of output?** This comes from **EXIOBASE 3.8**, a global multi-regional IO database that has WaterGAP-modelled water use data for 163 India sectors. We extract, convert, and map these to our 140-sector SUT framework.

The Water Coefficient Formula

```
Raw coefficient from EXIOBASE F.txt:

   W_i  =  F_water_i  /  x_i              (m³ per EUR million of output)

   where:  F_water_i  = water satellite row i (m³/year)
           x_i        = total output of sector i (EUR million)

Unit conversion to match Indian SUT (₹ crore):

   W_i [m³/₹ crore]  =  W_i [m³/EUR M]  ×  EUR_INR  /  10

   (÷10 because 1 EUR million = 0.1 EUR crore × EUR/INR rate = ₹ crore)
```

Blue Water vs Green Water

EXIOBASE's F.txt contains two distinct types of water rows:

Type	EXIOBASE Row Label	Physical Meaning	Role in This Study
Blue water	'Water Consumption Blue' (103 rows)	Surface + groundwater extracted and consumed (not returned to source)	PRIMARY metric — used in all TWF = W·L·Y calculations
Green water	'Water Consumption Green' (13 rows)	Rainfall absorbed by crop roots and transpired (rainfed agriculture only)	DISCLOSURE metric — reported separately, never added to blue in headline figures

Why green water only exists in agriculture

Green water = rainfall stored in soil that crops consume through their roots.
This physical process is unique to plants growing in soil.
Factories, hotels, power plants, and transport cannot consume green water.
EXIOBASE correctly assigns green coefficients to exactly 13 crop rows and zero to all others.
Reporting both gives the full hydrological burden; India's food system is ~60% rainfed.

Concordance: Mapping EXIOBASE 163 → SUT 140

EXIOBASE has 163 India sectors; our SUT has 140 products. They don't match exactly. A concordance table maps each EXIOBASE code to the SUT product(s) it corresponds to:

```
For each SUT product j that maps to EXIOBASE codes {e1, e2, ...}:

   W_sut[j]  =  average(W_exio[e1], W_exio[e2], ...)      (simple average)
```

```
or for demand-weighted concordance:
W_sut[j]  =  Σ(W_exio[e] × Y_exio[e]) / Σ(Y_exio[e])
```

Outputs from Step 2

- water_coefficients_163_{year}.csv — raw EXIOBASE coefficients, 163 sectors
- concordance_{year}.csv — EXIOBASE↔SUT sector mapping table
- water_coefficients_140_{year}.csv — SUT-aligned coefficients, 140 products
- Columns: Blue_Water_m3_per_crore, Green_Water_m3_per_crore, Product_ID, Product_Name

STEP 3 Build Tourism Demand Vectors [build_tourism_demand.py]

The **Tourism Satellite Account (TSA)** was only published for 2015-16. For 2019 and 2022, we extrapolate it forward using India's National Accounts Statistics (NAS) real GVA growth rates — the same approach used by the OECD and Temurshoev & Timmer (2011) for demand extrapolation when a base-year account is the only available source.

The Three-Stage Scaling Formula

```
Stage 1 — Real growth multiplier (from NAS Statement 6.1, constant 2011-12
prices):

   real_growth(sector, year)  =  GVA(sector, year) / GVA(sector, 2015-16)

Stage 2 — CPI adjustment (real → nominal, matching SUT data which is
nominal):

   CPI_multiplier(year)  =  CPI(year) / CPI(2015-16)

Stage 3 — Nominal tourism demand for each category c in year t:

   Demand_nominal(c, t)  =  Demand_TSA2015(c)  ×  real_growth(c, t)  ×
CPI_mult(t)

Separate vectors for inbound and domestic:
   Y_inbound(c, t)   =  Inbound_TSA2015(c)   × same scaling
   Y_domestic(c, t)  =  Domestic_TSA2015(c)  × same scaling
```

NAS Sector Mapping for Each TSA Category

The TSA has 24 categories (accommodation, food, transport, shopping, etc.). Each maps to a NAS GVA sector for the growth rate:

TSA Category	NAS Sector Used	Rationale
Accommodation (Hotels)	Hotels & Restaurants	Direct NAS category match
Food & Beverage	Hotels & Restaurants	Restaurants included in same NAS row
Rail transport	Railways	Direct NAS category match
Air transport	Air transport	Direct NAS category match
Shopping (goods)	Trade (Wholesale/Retail)	Retail margins on tourist purchases
Entertainment	Recreation & Cultural	NAS sub-row for entertainment services
All other	Other Services	Residual NAS services category

Mapping Tourism Categories to EXIOBASE Sectors

Once we have the 24-category demand values, we distribute them across 163 EXIOBASE sectors using TSA_TO_EXIOBASE shares (defined in reference_data.md). For example:

```
Category: 'Accommodation'  →  {'IN.55': 0.85, 'IN.56': 0.15}
```

```
  (85% to hotels/resorts, 15% to other accommodation)

Y_exio[55]  +=  Demand_Accommodation × 0.85
Y_exio[56]  +=  Demand_Accommodation × 0.15

Shares are normalised to sum = 1.0 per category before application.
```

Deflating to Real 2015-16 Prices

```
Y_real(year)  =  Y_nominal(year)  /  deflator(year)

deflator(year)  =  CPI(IO_year) / CPI('2015-16')

Example (2022):  deflator = CPI('2021-22') / CPI('2015-16')  =  1.47
→ Y_real_2022   = Y_nominal_2022 / 1.47   (₹ crore in 2015-16 prices)

Y_real is only used for intensity comparisons across years.
All TWF computations use Y_nominal to match the SUT data units.
```

Outputs from Step 3

- tsa_scaled_{year}.csv — 24-category TSA with nominal+real demand per year
- Y_tourism_{year}.csv — 163-sector EXIOBASE demand vector (nominal ₹ crore)
- Y_tourism_{year}_real.csv — same vector deflated to 2015-16 prices
- Y_tourism_{year}_inbound.csv — inbound-only demand vector
- Y_tourism_{year}_domestic.csv — domestic-only demand vector

STEP 4 Calculate Indirect Water Footprint (EEIO Core) [calculate_indirect_twf.py]

This is the **heart of the entire pipeline.** We combine the three inputs built in Steps 1–3 to compute how much water India's entire economy must use — upstream — to satisfy tourism demand. The answer captures water embedded in food processing, agriculture, manufacturing, energy, and every other supply-chain layer.

The Core EEIO Formula

```
TWF_indirect  =  W  ·  L  ·  Y          (m³ total)

where:
   W  =  water coefficient vector  (140,)   [m³ per ₹ crore]
          W[i] = blue water used per ₹ crore of product i output

   L  =  Leontief inverse matrix   (140×140)  [dimensionless]
          L[i,j] = total output of i required per unit of final demand for
j

   Y  =  tourism demand vector     (140,)   [₹ crore]
          Y[j] = total tourist spending on product j

Computation proceeds in two steps:
   WL[j]   =  W · L[:,j]   =  Σ_i  W[i] × L[i,j]   [m³ per ₹ crore of j
demand]
   TWF[j]  =  WL[j] × Y[j]                          [m³ from product j
demand]

Total:  TWF_total  =  Σ_j  TWF[j]
```

The Invisible Water Intuition

WL[j] is called the 'water multiplier' for sector j. It answers:
'Per rupee tourists spend on product j, how many cubic metres of water does the ENTIRE supply chain (all tiers) require?'

Example: WL[hotels] = 4,200 m³/₹ crore
→ Per ₹1 crore tourists spend on hotels, 4,200 m³ of water is mobilised through the supply chain — even though the hotel itself might use only 50 m³ directly (the rest is in food, laundry supplies, energy, etc.).

This is why indirect water >> direct water, and why agriculture — which tourists never buy directly — dominates the total.

Mapping Y from EXIOBASE 163 to SUT 140

Y is built in 163-sector EXIOBASE space (Step 3), but W and L are in 140-sector SUT space. We map using the concordance:

```
For each concordance row c mapping EXIOBASE codes {e1,...,ek} → SUT IDs
{s1,...,sm}:
```

```
   demand_c  =  Σ_e  Y_exio[e]          (sum EXIOBASE demand for this
category)
   Y_sut[s]  +=  demand_c / m           (split equally across m SUT
products)

Coverage check: Y_sut.sum() / Y_exio.sum() should be ≥ 99%
```

Scarce Water (WSI-Weighted)

Blue water counts extracted volume. Scarce water corrects for basin stress — a litre from Punjab (nearly empty) causes far more damage than a litre from Kerala (abundant):

```
Scarce_TWF[i]  =  TWF[i]  ×  WSI(source_group[i])

WSI values (WRI Aqueduct 4.0, Kuzma et al. 2023):
   Agriculture :  WSI = 0.827  (Aqueduct raw score 4.137/5.0)
   Manufacturing / Electricity / Mining / Petroleum:  WSI = 0.814
   Services :     WSI = 0.000  (municipal water — no direct basin
extraction)

Scarce/Blue ratio ≈ 0.83  →  for every 100 L extracted by tourism supply
chains,
83 L-equivalents of real basin damage is caused.
```

Water Multiplier Ratio

Beyond the raw WL value, we compute a ratio that shows whether tourism spending on each sector is more or less water-intensive than the average Indian economy:

```
Multiplier_Ratio[j]  =  WL[j]  /  economy_avg_WL

economy_avg_WL  =  Σ_j( WL[j] × Y_economy[j] ) / Σ_j( Y_economy[j] )
                   (demand-weighted average across all economy sectors)

Ratio > 1.0  →  sector j is more water-intensive than the economy average
               → high-priority target for water stewardship interventions
Ratio < 1.0  →  sector j is below-average water intensity
```

Sensitivity Analysis (±20% Shock)

```
For key upstream groups (Agriculture, Electricity, Petroleum):

   LOW:   TWF_low   = W_agriculture × 0.80  → (I−A)⁻¹ × Y
   BASE:  TWF_base  = W_agriculture × 1.00  → (I−A)⁻¹ × Y
   HIGH:  TWF_high  = W_agriculture × 1.20  → (I−A)⁻¹ × Y

Half-range ±%  =  (TWF_high − TWF_low) / (2 × TWF_base) × 100

Result:  ±20% agr. coefficient → ±14% TWF  (elasticity ≈ 0.71)
```

```
Reflects agriculture's ~70% share of upstream TWF origin.
```

Inbound vs Domestic Split

Using the separate demand vectors from Step 3:

```
TWF_inbound   = W · L · Y_inbound
TWF_domestic  = W · L · Y_domestic

TWF_total  ≈  TWF_inbound + TWF_domestic     (small rounding difference
only)

Intensity comparison:
   L_per_inbound_day  = TWF_inbound  × 1000 / tourist_days_inbound
   L_per_domestic_day = TWF_domestic × 1000 / tourist_days_domestic

Inbound/domestic ratio ≈ 10-18×  (spending basket, not infrastructure
difference)
```

Outputs from Step 4

- indirect_twf_{year}_by_sut.csv — 140-product results (TWF, Scarce, Multiplier_Ratio)
- indirect_twf_{year}_by_category.csv — 75-category aggregated results
- indirect_twf_{year}_origin.csv — water by source group (Agriculture/Mfg/Services)
- indirect_twf_{year}_sensitivity.csv — LOW/BASE/HIGH scenarios
- indirect_twf_{year}_split.csv — inbound vs domestic TWF
- water_multiplier_ratio_{year}.csv — WL[j] / economy_avg_WL per category
- indirect_twf_all_years.csv — cross-year summary

STEP 5 Calculate Direct (Operational) Water Footprint [calculate_direct_twf.py]

Direct water is water physically consumed **on-site** at tourism facilities: showers in hotels, water for cooking in restaurants, water for locomotive cooling on trains. Unlike the EEIO model, this is computed using **activity-based coefficients** from field studies and operational reports.

Direct TWF Formula

```
Hotel direct TWF:
   Hotel_m³  =  hotel_rooms_occupied  ×  365 days  ×  coeff_hotel
[L/room/night]  / 1000

Restaurant direct TWF:
   Rest_m³   =  tourist_meals_M  ×  1e6  ×  coeff_restaurant  [L/meal]  /
1000

Rail direct TWF:
   Rail_m³   =  rail_pkm_B  ×  1e9  ×  coeff_rail  [L/passenger-km]  / 1000

Air direct TWF:
   Air_m³    =  air_passengers_M  ×  1e6  ×  coeff_air  [L/passenger-
flight]  / 1000

Total direct (BASE):  Σ of all sectors
LOW / HIGH: same formula with low/high coefficients from reference_data.md
```

Coefficient Sources and Values

Sector	BASE coefficient	Unit	Source	Trend 2015→2022
Hotels (classified)	~330 L/room/night	L/room/night	CHSB India field studies 2015-2022	−34.6% (efficiency gains)
Restaurants	~25 L/meal	L/meal	Bohdanowicz & Martinac (2007) adapted	Stable
Rail	3.5 L/pass-km	L/passenger-km	Gössling (2015); IRCTC reports	Stable
Air	18 L/pass-flight	L/passenger-flight	Gössling (2015) aviation survey	Stable
Water transport	20 L/pass-trip	L/passenger-trip	Literature estimate	Stable

Why Direct TWF << Indirect TWF

Direct water (what hotels/restaurants/trains physically use) is typically < 5% of total blue TWF.
The other 95%+ is embedded in supply chains — food production, energy, textiles, packaging.
A hotel using 330 L/room/night on-site drives ~20,000 L/room/night through its entire supply chain.
This is the key policy implication: saving water at the tap matters far less than
changing what hotels buy (food sourcing, laundry services, energy supply).

STEP 6 Outbound TWF & Net Virtual Water Balance [outbound_twf.py]

When Indian tourists travel abroad, they consume water in destination countries — water that those (often water-stressed) countries must supply. This is India's **outbound water footprint**. Comparing it with inbound TWF gives the **net virtual water balance**: is India a net exporter or importer of water through tourism?

Outbound TWF Formula (Lee et al. 2021 Methodology)

```
For each destination country d:

   tourists_to_d  =  N_outbound_total  ×  dest_share_d

   daily_WF_d     =  local_WF_m³_yr_d  /  365
                     (Hoekstra & Mekonnen 2012 national per-capita WF)

   Outbound_m³_d  =  tourists_to_d  ×  avg_stay_days  ×  daily_WF_d  ×  1.5
                                                                       ↑
                     Tourist multiplier: tourists use 50% more water
                     than local residents per day (higher-quality accomm.,
                     more showers, pool use; Hadjikakou et al. 2015)

   Total outbound  =  Σ_d  Outbound_m³_d
```

Scarce Outbound Water

```
Outbound_Scarce_m³_d  =  Outbound_m³_d  ×  WSI_destination_d

WSI values per destination (Aqueduct 4.0):
   UAE          :  WSI = 1.00  (maximum stress — near complete depletion)
   Saudi Arabia:  WSI = 1.00  (maximum stress)
   USA          :  WSI = 0.35  (moderate stress)
   UK           :  WSI = 0.18  (low-moderate stress)

UAE/Saudi Arabia receive ~45% of Indian outbound tourists — India's
outbound tourism concentrates virtual water demand in the world's
most water-scarce basins.
```

Net Virtual Water Balance

```
Net_TWF  =  Outbound_TWF_total  −  Inbound_TWF_total

Positive net  →  India is a NET IMPORTER of water via tourism
                 (India sends more water to foreign destinations
                  than it receives from inbound tourists)

Negative net  →  India is a NET EXPORTER of water via tourism
                 (India hosts more tourism water use than it exports)
```

NOTE: These are NOT methodologically equivalent — outbound uses activity-based Lee et al. (2021); inbound uses EEIO Leontief W·L·Y. Net balance is therefore INDICATIVE ONLY, not a precise bilateral comparison.

STEP 7 SDA + Monte Carlo + Supply-Chain Analysis [calculate_sda_mc.py]

This step runs three complementary analyses that go beyond the point estimates:

- Structural Decomposition Analysis (SDA) — why did TWF change between years?
- Monte Carlo simulation — how uncertain is our estimate?
- Supply-chain path analysis — which specific chains carry the most water?

Part A: Structural Decomposition Analysis (SDA)

TWF changes between years because **W** changes (sectors became more/less water-efficient), **L** changes (supply-chain structure changed), and **Y** changes (tourism demand grew/shrank). SDA separates these three drivers — essential for asking which policy lever actually drove the change observed.

Two-Polar Dietzenbacher–Los Decomposition

```
Total change:  ΔTWF  =  TWF₁ − TWF₀  =  W₁L₁Y₁ − W₀L₀Y₀

Define:  ΔW = W₁ − W₀   ΔL = L₁ − L₀   ΔY = Y₁ − Y₀

Two-polar formula (Dietzenbacher & Los 1998):

  W-effect  =  0.5 × [ΔW·L₀·Y₀  +  ΔW·L₁·Y₁]
               (technology: water use per unit output changed)

  L-effect  =  0.5 × [W₀·ΔL·Y₀  +  W₁·ΔL·Y₁]
               (structure: supply-chain composition changed)

  Y-effect  =  0.5 × [W₀·L₀·ΔY  +  W₁·L₁·ΔY]
               (demand: tourism volume/composition changed)

Identity:  W-effect + L-effect + Y-effect  ≡  ΔTWF
Residual  =  0  (the two-polar decomposition is exact)
```

Interpreting the Effects

Effect	Sign	Meaning	Policy Lever
W (technology)	Negative (−)	Upstream sectors became more water-efficient per unit output	Irrigation efficiency; industrial water standards
W (technology)	Positive (+)	Upstream sectors became more water-intensive	Priority: address efficiency regression
L (structure)	Negative (−)	Supply chains shorter or less water-intermediated	Procurement standards; local sourcing
L (structure)	Positive (+)	More supply-chain layers; greater intermediation	Risk: multiplier growth amplifies future shocks
Y (demand)	Positive (+)	Tourism demand grew; more water mobilised	Demand management; low-water-intensity products

Effect	Sign	Meaning	Policy Lever
Y (demand)	Negative (−)	Tourism demand fell (COVID period)	Confirms demand-TWF correlation direction

The COVID 2019→2022 Finding

The L-effect (supply-chain restructuring) dominated the 2019→2022 ΔTWF, exceeding the Y-effect (demand contraction) by multiple times.

This means: DO NOT interpret the period as a demand-elasticity experiment. COVID simultaneously restructured India's tourism supply chains (shorter chains, different food sourcing, changed intermediation). The L-effect dominated because supply-chain structure is the PRIMARY lever for rapid TWF change — not how many tourists show up.

Part B: Monte Carlo Uncertainty Analysis

The EXIOBASE water coefficients carry substantial uncertainty — WaterGAP crop water use modelling has known uncertainty of ±30% at agricultural sector level. Monte Carlo propagates this through the full TWF calculation to produce a distribution of plausible outcomes.

Monte Carlo Design

```
For each simulation s (s = 1, ..., 10,000):

  1. Sample multiplier:  m_s ~ LogNormal(μ=0, σ=0.30)
     (log-normal because water use is strictly positive and right-skewed)

  2. Perturb all agricultural W coefficients simultaneously (single
correlated draw):
     W_agr_perturbed[s]  =  W_agr_base  ×  m_s
     (all 13 crop rows move together — conservative upper bound on
uncertainty)

  3. Compute perturbed TWF:  TWF[s] = W_perturbed[s] · L · Y

After 10,000 simulations, report:
  P5, P25, Median, P75, P95 of the TWF distribution
  Asymmetric CI: typically −X% / +Y% from base (Y > X due to log-normal
right skew)

Conservative design note: single correlated multiplier OVERSTATES
uncertainty.
Under independent sector sampling, partial cancellation reduces CI by ~30-
40%.
Reported CI is an upper bound; realistic uncertainty ≈ ±18-22%.
```

Variance Decomposition (Spearman Rank Correlation)

```
For each uncertain input parameter p:

  ρ_p  =  Spearman rank correlation(input_p across simulations, TWF across
simulations)

  Variance share_p  =  ρ_p²  ×  100%

Result: Agricultural W coefficient accounts for ~99% of Monte Carlo
variance
  → Improving WaterGAP crop-level estimates for India is the single
    highest-impact data improvement possible for this model.
  → Hotel/transport coefficient uncertainty: < 1% of total variance.
```

Part C: Supply-Chain Path Analysis

Which specific supply-chain routes carry the most water to tourism? We decompose the TWF matrix element by element to find the dominant pathways.

Leontief Pull Decomposition

```
For each source sector i and destination (tourism) sector j:

   pull[i,j]  =  W[i]  ×  L[i,j]  ×  Y[j]

   = water mobilised FROM sector i BY tourist spending ON sector j

Pull matrix is (140 × 140). Each element is a distinct supply-chain
pathway.
We rank all 140² = 19,600 elements and report the top 50.

Example pathway:  Agriculture → Food Manufacturing → Hotels
  pull[Paddy rice, Hotel]  =  W[paddy]  ×  L[paddy, hotel]  ×  Y[hotel]
  = this is the water in paddy rice irrigation that flows through food
    processors and ultimately enables hotel food service to tourists.
```

Hypothetical Extraction Method (HEM)

```
For each upstream sector i:

  TWF_with_i      = W · L · Y          (normal calculation)
  TWF_without_i   = W · L · Y  but Y[i] = 0  (extract sector i from demand)

  Dependency_Index[i]  =  (TWF_with_i − TWF_without_i) / TWF_with_i × 100%

High Dependency_Index → sector i's output is heavily purchased by tourism
and carries high water intensity. Most water-dependent sectors are
typically paddy/wheat farming and food processing.
```

STEP 8 Visualise Results [visualise_results.py]

This step reads all computed CSVs and generates the eight figures for the manuscript. No new formulas — all computations were done in Steps 1–7.

Figure	Type	What It Shows	Key Formula / Data Source
Figure 1	Flow diagram	Analytical framework — all 6 pipeline phases	Visual only
Figure 2	Four-panel overview	All four key metrics in one plate (volumes, intensity, hotspots, source composition)	Tables 4, 7a, 7d, 24
Figure 3	Temporal trajectory	TWF over time, intensity gap, demand composition	indirect_twf_all_years.csv + split CSVs
Figure 4	Heatmap (WMR)	Water multiplier ratio by sector and year — hotspot + artefact audit	water_multiplier_ratio_{year}.csv
Figure 5	Bubble matrix	Leontief pull: source sector × demand sector × water volume	sc_paths_{year}.csv
Figure 6	Sankey diagram	Source → supply chain → tourism demand water flow	indirect_twf_{year}_structural.csv
Figure 7	Waterfall chart	SDA decomposition W/L/Y effects with COVID annotation	sda_summary_all_periods.csv
Figure 8	Uncertainty strip	MC P5–P95 band per year; asymmetric CI; dominant driver	mc_summary_all_years.csv

STEP 9 Compare Years & Generate Report [compare_years.py]

The final step reads all result CSVs, computes cross-year comparisons and growth rates, fills every {{PLACEHOLDER}} in report_template.md, and writes the final Markdown report.

Key derived calculations in this step

Metric	Formula
CAGR between periods	CAGR = ((TWF_end / TWF_start)^(1/n_years) − 1) × 100%
Per-tourist-day intensity	L/day = TWF_total_m³ × 1000 / tourist_days
Intensity decline %	(Intensity_last − Intensity_first) / Intensity_first × 100%
Inbound/domestic ratio	L_per_day_inbound / L_per_day_domestic
SDA L/Y ratio	|L-effect_m³| / |Y-effect_m³|
MC CI half-width %	(P95 − P5) / (2 × BASE) × 100%

How All Steps Connect: The Full Computation Chain

How All Steps Connect: The Full Computation Chain

The Single Equation That Unifies Everything

Every result in this pipeline flows from one equation:

$$TWF = W \cdot (I - A)^{-1} \cdot Y$$

W comes from Step 2 (EXIOBASE water satellites)
$(I - A)^{-1}$ = L comes from Step 1 (SUT → IO table → Leontief inverse)
Y comes from Step 3 (TSA × NAS growth × CPI)

Steps 4-9 are all extensions, refinements, and validations of this product:
- Step 4: compute it, decompose by sector, weight by WSI
- Step 5: add direct (activity-based) component separately
- Step 6: apply same logic to outbound tourist flows
- Step 7: decompose $\Delta TWF = \Delta W \cdot L \cdot Y + W \cdot \Delta L \cdot Y + W \cdot L \cdot \Delta Y$ and quantify uncertainty
- Steps 8-9: visualise and report

Data flow diagram (text)

```
MoSPI SUTs (2015-16, 2019-20, 2021-22)
    └─ build_io_tables.py ──────────────▶ L matrix (140×140, per year)

EXIOBASE 3.8 F.txt + x vector
    └─ build_water_coefficients.py ──▶ W vector (140, per year)

India TSA 2015-16 + NAS Statement 6.1 + CPI
    └─ build_tourism_demand.py ─────────▶ Y vector (163 → 140, per year)
                                           Y_inbound, Y_domestic

W + L + Y ──────────────────────────────────▶
    └─ calculate_indirect_twf.py ──────▶ TWF_indirect (bn m³, per year)
                                           Scarce TWF (WSI-weighted)
                                           WL multipliers, sensitivity

Activity data + field coefficients
    └─ calculate_direct_twf.py ────────▶ TWF_direct (LOW/BASE/HIGH, per
year)

Outbound counts + dest. WF data
    └─ outbound_twf.py ──────────────── Outbound m³ + net balance

TWF years 0,1  → SDA decomposition:  ΔTWF = ΔW + ΔL + ΔY
W uncertainty  → Monte Carlo:         P5–P95 distribution
pull[i,j]      → Supply-chain paths:  top 50 dominant chains
    └─ calculate_sda_mc.py ──────────▶ All three outputs

All CSVs ──────────────────────────────────▶
```

```
    └─ visualise_results.py ──────────► Figures 1–8
    └─ compare_years.py ──────────────► Final Markdown report
```

Key Numbers to Remember

What	Value	Where it comes from
Core formula	TWF = W · L · Y	EEIO theory; Leontief (1970)
SDA formula	ΔTWF = ΔW·L·Y + W·ΔL·Y + W·L·ΔY	Dietzenbacher & Los (1998)
Scarce water	Scarce = Blue × WSI (0.827 agr.)	Aqueduct 4.0, Kuzma et al. (2023)
Outbound formula	N × days × (WF/365) × 1.5	Lee et al. (2021)
Agriculture WSI	0.827 (Aqueduct 4.137/5)	Near-maximum global water stress
MC design	LogNormal(μ=0, σ=0.30) on agr. W	Biemans et al. (2011)
Tourist multiplier	1.5× local per-capita WF	Hadjikakou et al. (2015)
IO method	Product Technology Assumption	OECD/Eurostat standard
Agr. share of indirect TWF	~70–85% across all years	Supply-chain propagation via L
Direct % of total TWF	< 5%	Activity-based vs EEIO comparison

India Tourism Water Footprint Pipeline — Technical Methodology Document
Framework: Leontief (1970) · Miller & Blair (2009) · Hoekstra et al. (2011) · Dietzenbacher & Los (1998)